AI Image Captioning App

with BLIP and Gradio

Author	Jack Pumpuni Frimpong-Manso
Affiliation	Environmental Data Scientist | Guest Scientist, ZMT Bremen

Date	2026
Source	IBM Skills Network — Guided Project

Table of Contents

1. Introduction and Objectives

Images contain rich information that standard data systems and search engines cannot easily interpret. Converting visual content into machine-readable text — image captioning — bridges this gap and unlocks a broad set of practical applications across industry, research, and accessibility.

Why Image Captioning Matters

Benefit	Description
Accessibility	Helps visually impaired users understand visual content via screen readers
SEO Enhancement	Enables search engines to index image content, driving organic traffic
Content Discovery	Allows efficient analysis and categorisation of large image databases
Social Media	Automates engaging description generation for visual content
Security	Provides real-time descriptions of activities in video footage
Education	Assists in interpreting and documenting visual materials
Multilingual	Captions can be generated in multiple languages for global audiences
Data Organisation	Helps manage and categorise large sets of visual data
Time Efficiency	Automated captioning is significantly faster than manual efforts
User Engagement	Detailed, accurate captions make visual content more informative

Learning Objectives

By the end of this project, you will be able to:

- Implement an image captioning tool using the **BLIP model** from Hugging Face's Transformers library
- Use **Gradio** to build a user-friendly web interface for the captioning application
- Apply the tool in a real-world **news and media business scenario**
- Adapt the code for **automated URL-based captioning** of entire web pages

2. Environment Setup

Step 1 — Create and activate a virtual environment

```
pip3 install virtualenv
virtualenv my_env
source my_env/bin/activate # Linux / macOS
# my_env\Scripts\activate # Windows
```

Step 2 — Install all required libraries

```
pip install langchain==0.1.11 \
gradio==5.23.2 \
transformers==4.38.2 \
bs4==0.0.2 \
requests==2.31.0 \
torch==2.2.1
```

Note: Installation takes approximately 5 minutes. The BLIP model weights (~990 MB) are downloaded automatically on first run.

Library Reference

Library	Version	Purpose
transformers	4.38.2	Hugging Face — BLIP processor and model
gradio	5.23.2	Web interface for the captioning app
torch	2.2.1	PyTorch backend for model inference
Pillow	—	Image loading and RGB conversion
requests	2.31.0	HTTP requests for URL-based scraping
bs4	0.0.2	BeautifulSoup — HTML parsing for URL captioner
langchain	0.1.11	LangChain framework (available for extension)

3. The BLIP Model — How It Works

What is Hugging Face Transformers?

Hugging Face is an organisation focused on natural language processing (NLP) and AI, widely known for its open-source *transformers* library. The library provides thousands of pre-trained models for tasks including translation, summarisation, text generation, and vision-language understanding. It has made state-of-the-art models (BERT, GPT-2, GPT-3, and others) accessible to researchers and developers worldwide.

What is BLIP?

BLIP (Bootstrapping Language-Image Pre-training) is a vision-language model that enables computers to understand and generate natural language descriptions based on images. Pre-trained on a large corpus of image-text pairs, BLIP can perform image captioning, visual question answering (VQA), and image-text retrieval. The model used in this project is **Salesforce/blip-image-captioning-base**, available on the Hugging Face model hub.

Key Components

AutoProcessor

The processor wraps a BLIP image processor and an OPT/T5 tokenizer into a single object. It handles both image preprocessing and text tokenization, preparing inputs for the model in the correct tensor format. A *tokenizer* is a tool in NLP that breaks text into smaller units (tokens) — such as words or subwords — so that models can analyse and understand the text.

BlipForConditionalGeneration

This model class performs conditional text generation given an image and an optional text prompt. It generates a sequence of tokens that are decoded into human-readable text, making it suitable for image captioning and visual question answering tasks.

Inference Pipeline

Stage	Component	Output
1. Input	Local file or URL image	Raw image data
2. Preprocess	AutoProcessor	Pixel tensors + input_ids
3. Inference	BlipForConditionalGeneration.generate()	Token ID sequence
4. Decode	processor.decode(skip_special_tokens=True)	Caption string

4. Code Walkthrough

4.1 Script 1 — Single Image Captioning (image_cap.py)

This script demonstrates the core captioning pipeline applied to a single local image. It is the foundational building block on which the other two scripts are based.

```
import requests
from PIL import Image
from transformers import AutoProcessor, BlipForConditionalGeneration

# Load the pretrained processor and model
processor = AutoProcessor.from_pretrained("Salesforce/blip-image-captioning-base")
model = BlipForConditionalGeneration.from_pretrained("Salesforce/blip-image-captioning-base")

# Load and preprocess the image
img_path = "YOUR_IMAGE_NAME.jpeg"
image = Image.open(img_path).convert('RGB')

# Prepare inputs for the model
text = "the image of"
inputs = processor(images=image, text=text, return_tensors="pt")

# Generate the caption
outputs = model.generate(**inputs, max_length=50)

# Decode tokens to human-readable text
caption = processor.decode(outputs[0], skip_special_tokens=True)
print(caption)
```

Key Design Decisions

Decision	Reason
convert('RGB')	Ensures 3-channel input regardless of source format (RGBA, greyscale)
return_tensors='pt'	Returns PyTorch tensors required by the BLIP model
**inputs	Unpacks the processor dictionary as keyword arguments to model.generate()
skip_special_tokens=True	Removes model control tokens from the output string
max_length=50	Caps caption length at 50 tokens for concise, readable output

4.2 Script 2 — Automated URL Captioner (automate_url_captioner.py)

This script automatically scrapes all images from a given webpage URL and writes a caption for each image to a captions.txt output file. BeautifulSoup parses the HTML and extracts image URLs from src, data-src, and srcset attributes.

```
import requests
from PIL import Image
from io import BytesIO
from bs4 import BeautifulSoup
```

```
from transformers import AutoProcessor, BlipForConditionalGeneration

processor = AutoProcessor.from_pretrained("Salesforce/blip-image-captioning-base")
model = BlipForConditionalGeneration.from_pretrained("Salesforce/blip-image-captioning-base")

url = "https://en.wikipedia.org/wiki/IBM"
headers = {"User-Agent": "Mozilla/5.0"}
response = requests.get(url, headers=headers)
soup = BeautifulSoup(response.text, "html.parser")
img_elements = soup.find_all("img")

with open("captions.txt", "w", encoding="utf-8") as caption_file:
for idx, img_element in enumerate(img_elements, start=1):
img_url = img_element.get("src") or img_element.get("data-src")
if not img_url and img_element.has_attr("srcset"):
img_url = img_element["srcset"].split()[0]
if not img_url: continue
if img_url.endswith(".svg") or ".svg" in img_url: continue
if img_url.startswith("//"): img_url = "https:" + img_url
elif img_url.startswith("/"): img_url = "https://en.wikipedia.org" + img_url
elif not img_url.startswith("http"): continue
try:
r = requests.get(img_url, timeout=10, headers=headers)
raw_image = Image.open(BytesIO(r.content))
if raw_image.size[0] * raw_image.size[1] < 200: continue
raw_image = raw_image.convert("RGB")
text = "the image of"
inputs = processor(images=raw_image, text=text, return_tensors="pt")
out = model.generate(**inputs, max_new_tokens=50)
caption = processor.decode(out[0], skip_special_tokens=True)
caption_file.write(f"{img_url}: {caption}\n")
except OSError: continue
except Exception as e: print(f'[{idx}] Error: {e}'); continue
```

Key Engineering Decisions

Decision	Reason
User-Agent header	Prevents request blocking by servers that reject bot traffic
SVG skip logic	PIL cannot open SVG files; skipping avoids OSError crashes
Pixel area < 200 threshold	Eliminates tracking pixels, spacer GIFs, and tiny icons
srcset fallback	Modern responsive images often omit src and use srcset instead
BytesIO wrapping	Allows PIL to open image bytes without writing to disk
except OSError separately	Silently skips unreadable images; others still print for debugging

4.3 Script 3 — Gradio Web App (image_captioning_app.py)

This script wraps the BLIP captioning pipeline in a Gradio web interface, allowing any user to upload an image and receive a generated caption through a browser — no command-line knowledge required.

```
import gradio as gr
import numpy as np
from PIL import Image
from transformers import AutoProcessor, BlipForConditionalGeneration

processor = AutoProcessor.from_pretrained("Salesforce/blip-image-captioning-base")
model = BlipForConditionalGeneration.from_pretrained("Salesforce/blip-image-captioning-base")

def caption_image(input_image: np.ndarray):
# Gradio passes images as NumPy arrays; convert to PIL RGB
raw_image = Image.fromarray(input_image).convert('RGB')
text = "the image of"
inputs = processor(images=raw_image, text=text, return_tensors="pt")
outputs = model.generate(**inputs, max_length=50)
caption = processor.decode(outputs[0], skip_special_tokens=True)
return caption

iface = gr.Interface(
fn=caption_image,
inputs=gr.Image(),
outputs="text",
title="Image Captioning",
description="A simple web app for generating captions using the BLIP model."
)
iface.launch()
```

Gradio Interface Parameters

Parameter	Value	Purpose
fn	caption_image	The function to wrap with a UI
inputs	gr.Image()	Image upload component; returns NumPy array to the function
outputs	'text'	Text display component for the generated caption
title	Image Captioning	Displayed at the top of the web page
description	String	Instructional text shown below the title

Note: Gradio passes uploaded images as NumPy arrays to the function, which is why Image.fromarray(input_image) is needed before processing — unlike Script 1 which loads directly from a file path.

5. Business Scenario and Use Cases

News and Media Agency

A news agency publishing hundreds of articles daily faces a significant bottleneck when manually writing image captions for every visual asset. Integrating this captioning pipeline into the editorial workflow transforms the process:

Step	Action
1	Journalist writes article and selects relevant images
2	Images are fed into the captioning program (URL captioner or Gradio app)
3	Captions are automatically generated for every image
4	Editor reviews and approves or refines the captions
5a (Accessibility)	Captions added as alt text — screen reader users understand every image
5b (SEO)	Keyword-rich alt text indexed by Google — drives organic search traffic
6	Article published with all images fully captioned

Additional Industries

Industry	Application
E-commerce	Auto-generate product descriptions from product images
Healthcare	Assist in documenting and labelling medical images for records
Security & Surveillance	Real-time textual description of CCTV footage events
Social Media Management	Generate post descriptions for scheduled visual content
Archives & Libraries	Automatically catalogue and tag historical photo collections
Education	Generate descriptive labels for scientific and educational images

6. Results

Script Outputs

image_cap.py — Single image output

The model receives a local image and a text prompt and produces a natural language caption printed to the terminal. Example:

```
the image of a dog sitting on a wooden floor next to a window
```

automate_url_captioner.py — Batch URL output

All images found on the target webpage are captioned and written to captions.txt. Each line follows the format:

```
<image_url>: <generated caption>
```

```
# Example:
```

```
https://upload.wikimedia.org/.../IBM_logo.png: the image of a blue ibm logo on a white background
```

```
https://upload.wikimedia.org/.../IBM_hq.jpg: the image of a large blue glass building
```

image_captioning_app.py — Gradio web interface

The application launches on http://localhost:7860. Users interact through an image upload box (drag-and-drop or file browser) and a text output field. No command-line interaction is required after launch.

Model Performance Notes

- The **blip-image-captioning-base** model generates grammatically coherent captions for most natural photographs, illustrations, and screenshots.
- Performance is strongest on images with clear subjects (people, objects, landscapes) and weaker on abstract graphics, charts, and diagrams.
- Caption quality improves with higher-resolution input images.
- The optional **Salesforce/blip2-opt-2.7b** (BLIP-2) model offers significantly improved captioning quality at the cost of approximately 10 GB of storage and higher compute requirements.

7. Conclusions and Next Steps

Key Takeaways

- The **BLIP model** from Hugging Face provides a powerful, ready-to-use vision-language pipeline that requires no training from scratch.
- **Gradio** makes deploying an ML model as an interactive web application a matter of a few lines of code, dramatically lowering the barrier to sharing AI tools.
- **BeautifulSoup** enables scalable automation: rather than captioning images one at a time, the URL captioner processes entire web pages in a single run.
- The pipeline is modular — each script builds on the same processor/model core and can be adapted independently for different deployment contexts.

Limitations

- The dataset used to pre-train BLIP is not domain-specific; captions for highly specialised imagery (medical scans, satellite images, engineering schematics) may be generic.
- The current URL captioner operates synchronously; processing pages with many images is slow. Asynchronous or parallel downloading would significantly improve throughput.
- The Gradio app runs locally by default; making it publicly accessible requires deployment to a cloud platform.
- The *base* model variant prioritises speed over accuracy; the *large* or BLIP-2 variants produce richer captions but require more compute resources.

Next Steps

Step	Action	Benefit
1	Deploy to IBM Code Engine or Hugging Face Spaces	Permanent public URL for the Gradio app
2	Upgrade to BLIP-2 (blip2-opt-2.7b)	Higher-quality captions on complex images
3	Add multilingual output via MarianMT	Captions in multiple languages for global audiences
4	Add parallel downloading with concurrent.futures	Faster batch processing for large web pages
5	Fine-tune on a domain-specific dataset	Improved caption relevance for specialised fields
6	Wrap captioning function in a FastAPI endpoint	CMS integration for automated editorial workflows

References

- Hugging Face BLIP Model: Salesforce/blip-image-captioning-base (huggingface.co)
- Hugging Face Transformers Documentation: huggingface.co/docs/transformers
- Gradio Documentation: gradio.app/docs
- Li et al. (2022). BLIP: Bootstrapping Language-Image Pre-training. ICML 2022.
- IBM Skills Network — Give Meaningful Names to Your Photos with AI (Guided Project)

8. Deployment with IBM Code Engine

IBM Code Engine is a fully managed, serverless platform that runs containerised workloads — web apps, microservices, event-driven functions, and batch jobs — all hosted within the same Kubernetes infrastructure. It lets you focus entirely on writing code, not on managing servers.

8.1 Required Files

File	Purpose
demo.py	Gradio application — the Python script that launches the web interface
requirements.txt	All Python library dependencies the app needs
Dockerfile	Blueprint that instructs the container runtime how to assemble the image

```
mkdir myapp && cd myapp
touch demo.py Dockerfile requirements.txt
```

8.2 File Contents

requirements.txt

```
requests
gradio==5.23.2
```

demo.py

```
import gradio as gr

def greet(name, intensity):
return "Hello, " + name + "!" * int(intensity)

demo = gr.Interface(
fn=greet,
inputs=["text", "slider"],
outputs=["text"],
)
demo.launch(server_name="0.0.0.0", server_port=7860)
```

Note: Test locally with: python3 demo.py — app runs at http://0.0.0.0:7860/

Dockerfile

```
FROM python:3.10

WORKDIR /app
COPY requirements.txt requirements.txt
```

```
RUN pip3 install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "demo.py"]
```

Instruction	Explanation
FROM python:3.10	Inherits the official Python 3.10 base image with all Python tooling
WORKDIR /app	Sets /app as the default working directory for all subsequent commands
COPY requirements.txt	Copies the dependencies file into the image before installing
RUN pip3 install ...	Installs all listed dependencies into the image (no cache = smaller image)
COPY . .	Copies all remaining source files including demo.py into the image
CMD ["python", "demo.py"]	Defines the command that runs when the container starts

8.3 Creating the Code Engine Project

IBM Code Engine projects group applications, jobs, and builds together and manage resource access. In the IBM Skills Network environment a project is pre-configured. Launch the Code Engine CLI from the IDE and verify your project:

```
ibmcloud ce project get --name "YOUR_PROJECT_NAME"
```

8.4 Building the Container Image

Step 1 — Navigate to source directory

```
cd myapp
```

Step 2 — Create the build configuration

```
ibmcloud ce build create --name build-local-dockerfile1 \
--build-type local --size large \
--image us.icr.io/${SN_ICR_NAMESPACE}/myapp1 \
--registry-secret icr-secret
```

Argument	Value	Purpose
--name	build-local-dockerfile1	Name of the build configuration
--build-type	local	Source is the local directory, not a Git repo
--size	large	More CPU, memory, and disk for large model pipelines
--image	us.icr.io/${SN_ICR_NAMESPACE}/myapp1	Target registry and image tag
--registry-secret	icr-secret	Credentials for pushing to IBM Cloud Container Registry

Step 3 — Submit and run the build

```
ibmcloud ce buildrun submit --name buildrun-local-dockerfile1 \
--build build-local-dockerfile1 \
--source .
```

Note: Allow approximately 3–5 minutes for the build to complete.

Step 4 — Monitor build progress

```
ibmcloud ce buildrun get -n buildrun-local-dockerfile1
```

When Status shows Succeeded, the container image has been built and pushed to your registry namespace.

8.5 Deploying the Containerised Application

```
ibmcloud ce application create --name demo1 \
--image us.icr.io/${SN_ICR_NAMESPACE}/myapp1 \
--registry-secret icr-secret \
--es 2G \
--port 7860 \
--minscale 1
```

Argument	Value	Reason
--name	demo1	Name of the deployed application
--image	us.icr.io/.../myapp1	Container image to pull and run
--registry-secret	icr-secret	Registry access credentials
--es	2G	Ephemeral storage — 2 GB needed for large model files
--port	7860	Gradio's default port; public traffic is routed here
--minscale	1	Keeps at least one instance running; prevents cold-start delays

Note: Allow 3–5 minutes for the deployment to complete.

8.6 Accessing the Deployed Application

Retrieve the public URL of your running app:

```
ibmcloud ce app get --name demo1 --output url
```

Open the returned URL in any browser to access the live Gradio application. The app is publicly accessible without any local server running. For a custom domain, IBM Code Engine supports Cloudflare-based domain configuration.

8.7 Deployment Architecture Summary

Stage	Command / Component	Action
1. Source	myapp/ directory	demo.py + requirements.txt + Dockerfile ready locally
2. Build config	ibmcloud ce build create	Define image name, registry, and build type
3. Build run	ibmcloud ce buildrun submit	Pack source, upload, and build container image
4. Registry	IBM Cloud Container Registry	Stores the built container image in your namespace
5. Deploy	ibmcloud ce application create	Pull image, start container, expose on port 7860
6. Access	ibmcloud ce app get --output url	Public HTTPS URL accessible from any browser